AI Intern Assignment 2: The Policy That Changed

A grounded question-answering assistant over conflicting documents - 60 minutes

Ground rules

Item	Details
Time	60 minutes of working time. Start the clock when you open this PDF. Push before the hour ends; commits after the deadline are ignored.
Submit	A GitHub repo link (public, or private with access granted to the reviewer). It must have at least 4 commits spread over the hour.
Required files	**README.md** (how to run, what works, what doesn't), **DECISIONS.md** (every ambiguity you hit and what you chose and why), **AI_USAGE.md** (honest log of any AI tool use, or 'none').
AI tools	Allowed, but you must disclose use in AI_USAGE.md. Be warned: the spec contains traps, contradictions and messy data that copy-pasting will not survive, and you should be ready to explain every part of your code and the decisions behind it in a follow-up discussion.
Scoring	Judgement and correctness on tricky cases beat feature count. A small, correct, well-reasoned solution beats a large, shaky one. Unfinished is fine if you say so honestly.

Scenario

Our support team is flooded with merchant questions about fees, refunds and limits. Build a small question-answering assistant over our policy documents in **ai-2-data/**: policy_master_v1.md, circular_2026_08_amendment.md and merchant_faq_2026-03.md. The documents are imperfect on purpose: they were written at different times and do not always agree.

What you must build

- `ask(question, as_of_date)` returning: **answer**, **citations** (document + clause number), and a **status**: ANSWERED, CONFLICT_RESOLVED (and how), CANNOT_ANSWER.
- The answer must depend on **as_of_date** - the same question asked for 20 August 2026 and 5 September 2026 may have different correct answers.
- Arithmetic questions must be calculated by **code**, not guessed by a language model.
- You may use an LLM, embeddings, plain keyword search, or hand-written rules. It must run for a reviewer without an API key (offline mode, or committed cached responses). Never invent a clause number.

Questions your system must handle (we will also ask others)

Unless a question gives a date, use as_of_date = 15 August 2026.

- Q1. When does a small merchant receive UPI money? (answer on 15 Aug 2026)
- Q2. A large merchant (above Rs.10 lakh monthly) receives a Rs.4,000 UPI payment. What MDR and GST do they pay?
- Q3. Same merchant, a Rs.1,50,000 payment. What do they pay?
- Q4. A merchant refunds a customer 3 hours after the sale. Do they get the MDR back? Ask it for 20 Aug 2026 and again for 5 Sep 2026.
- Q5. What does the FAQ say about the MDR cap, and is the FAQ right?
- Q6. Can a customer pay Rs.1,00,000 twice in one day to a merchant?

- Q7. I closed my account yesterday. When will I get my money?
- Q8. Does the policy allow me to settle card payments same day?

Write your own evaluation

- **eval.json** with at least 12 questions you wrote, each with expected answer, expected citations and as_of_date. At least 4 must be adversarial: questions designed to make a naive system fail (stale FAQ, deleted clause, missing information, false premise).
- A script that runs your system on eval.json and prints a score table.
- **WHERE_IT_BREAKS.md**: three questions your system gets wrong or shouldn't be trusted with, and why.

What we look at

Area	We look for
Document reasoning	Noticing superseded clauses, effective dates, the stale FAQ, exceptions and cross-references
Grounding	Real citations; says CANNOT_ANSWER instead of guessing
Evaluation	Quality and nastiness of your own test questions
Engineering	Deterministic arithmetic; works offline; clean structure